Study Notes: Structuring a `Student` Type Using Private Data, Getters, Setters, Constructors, and Multiple Files

1. Starting Point: Grouping Student Data Together

Earlier, we saw that a `struct` can be used to group related pieces of data under one name. For example, a student may have a name, an age, and some pocket money. Instead of keeping these as three separate variables in `main()`, we can group them inside a single `Student` type.

```
struct Student
{
    std::string name;
    int age;
    double pocket_money;
};
```

This already improves the program because the data now belongs together logically. We are no longer treating `name`, `age`, and `pocket_money` as unrelated variables. They describe one real-world concept, a student.

However, there is still a problem. If all these members are directly accessible from outside the structure, then any part of the program can change them freely.

```
Student s1;

s1.name = "Alice";
s1.age = -20;
s1.pocket_money = -500.0;
```

Technically, the compiler allows this if the members are public. But logically, this may not make sense. A student should not have a negative age, and pocket money should probably not be assigned carelessly from anywhere in the program.

So the next improvement is to stop outside code from touching the internal data directly.

2. Hiding the Member Variables Using `private`

To prevent direct access from outside the `Student` type, we place the member variables under the `private` access specifier.

```
struct Student
{
```

```
  private:
    std::string name;
    int age;
    double pocket_money;
};
```

Once the member variables are private, they cannot be accessed directly from `main()`.

```
Student s1;

s1.name = "Alice";          // Not allowed
s1.age = 20;                // Not allowed
s1.pocket_money = 100.50;   // Not allowed
```

This is an important design step. The data still exists inside the `Student` object, but outside code can no longer freely modify it. The object is now protecting its internal state.

At this stage, the program becomes safer, but we have also introduced a new problem. If outside code cannot directly access `name`, `age`, and `pocket_money`, how do we give values to a student object? Also, how do we read those values later?

The answer is to provide selected public functions.

3. Providing Controlled Access Using Setter Functions

A setter function is a public method that allows outside code to set the value of a private member variable.

For example:

```
void set_name(const std::string &name_str);
void set_age(int age_val);
void set_pocket_money(double money_val);
```

These functions are public, so they can be called from `main()`.

```
Student s1;

s1.set_name("Alice");
s1.set_age(20);
s1.set_pocket_money(100.50);
```

Now the outside world is no longer changing the member variables directly. Instead, it is making a request through a method provided by the `Student` type.

This gives the programmer control over how the data is changed. For example, later we could decide that `set_age()` should reject negative values, or that `set_pocket_money()` should not accept unrealistic values. The important point is that the decision is now controlled inside the `Student` implementation, not scattered randomly across the whole program.

The method definition may look like this:

```
void Student::set_age(int age_val)
{
    age = age_val;
}
```

Here, `Student::set_age` means that this function belongs to the `Student` type. The `age` member is private, but the method is part of the same type, so it is allowed to access it.

4. Providing Read Access Using Getter Functions

After setting the values, we also need a clean way to read them. For that, we use getter functions.

```
std::string get_name() const;
int get_age() const;
double get_pocket_money() const;
```

These methods return the value of the private member variables.

```
std::cout << "Name: " << s1.get_name() << std::endl;
std::cout << "Age: " << s1.get_age() << std::endl;
std::cout << "Pocket Money: $" << s1.get_pocket_money() << std::endl;
```

The getter method definition may look like this:

```
int Student::get_age() const
{
    return age;
}
```

The `const` at the end means this method does not modify the object. A getter should normally only read data, not change it. Therefore, marking it as `const` makes the intention clear and also allows the compiler to enforce that rule.

So now we have a clear design:

```
struct Student
{
  public:
```

```
    void set_name(const std::string &name_str);
    void set_age(int age_val);
    void set_pocket_money(double money_val);

    std::string get_name() const;
    int get_age() const;
    double get_pocket_money() const;

  private:
    std::string name;
    int age;
    double pocket_money;
};
```

The public section contains what outside code is allowed to use. The private section contains the internal data that outside code is not allowed to touch directly.

5. Initialising Member Variables Using a Default Constructor

When an object is created, its member variables should ideally start with known values.

For example, we may want a default student to begin like this:

```
name = "";
age = 0;
pocket_money = 0.0;
```

A default constructor is a special method that runs automatically when an object is created without arguments.

```
Student::Student()
{
    name = "";
    age = 0;
    pocket_money = 0.0;
}
```

Now, when we write:

```
Student s1;
```

the constructor runs automatically and prepares the object with sensible starting values.

This is important because uninitialised built-in values such as `int` and `double` may otherwise contain unpredictable values. A good object should begin life in a valid and understandable state.

6. Using `= default` in Modern C++

Modern C++ allows us to ask the compiler to generate a default constructor for us.

```
Student() = default;
```

This means: create the normal default constructor automatically.

However, there is an important detail. A compiler-generated default constructor does not automatically set built-in members such as `int` and `double` to zero in every situation. Therefore, if we want guaranteed known values, it is better to give the members default values directly where they are declared.

```
struct Student
{
  public:
    Student() = default;

    void set_name(const std::string &name_str);
    void set_age(int age_val);
    void set_pocket_money(double money_val);

    std::string get_name() const;
    int get_age() const;
    double get_pocket_money() const;

  private:
    std::string name{};
    int age{};
    double pocket_money{};
};
```

Now the compiler-generated constructor is still used, but the member variables also have clear default values.

This is a clean modern C++ style. It keeps the code simple while ensuring that the object starts in a known state.

7. Using the `Student` Type from `main()`

After the `Student` type has been designed, `main()` no longer works directly with the internal data. It only uses the public methods.

```
#include "student.h"
#include <iostream>
#include <string>
```

```
int main()
{
    Student s1;

    s1.set_name("Alice");
    s1.set_age(20);
    s1.set_pocket_money(100.50);

    std::cout << "Name: " << s1.get_name() << std::endl;
    std::cout << "Age: " << s1.get_age() << std::endl;
    std::cout << "Pocket Money: $" << s1.get_pocket_money() << std::endl;

    return 0;
}
```

This is a much better structure than directly writing:

```
s1.age = 20;
```

The user of the `Student` type does not need to know how the data is stored internally. They only need to know which public methods are available.

This is similar to using a device through buttons or a menu. The user does not need access to the internal wiring. They only need the safe controls provided by the designer.

8. Separating the Declaration into `student.h`

As the program grows, keeping everything inside `main.cpp` becomes difficult. So we separate the `Student` declaration into a header file.

The header file tells other files what the `Student` type looks like and what methods are available.

```
#ifndef STUDENT_H
#define STUDENT_H

#include <string>

struct Student
{
  public:
    Student() = default;

    void set_name(const std::string &name_str);
    void set_age(int age_val);
    void set_pocket_money(double money_val);

    std::string get_name() const;
    int get_age() const;
```

```
    double get_pocket_money() const;

  private:
    std::string name{};
    int age{};
    double pocket_money{};
};

#endif
```

This file does not contain the full method logic. It mainly contains the declaration of the type and the list of methods that other parts of the program are allowed to use.

The header file acts like a public description of the `Student` type.

9. Keeping Method Definitions in `main.cpp`

At first, we may keep the method definitions in `main.cpp` after including the header file.

```
#include "student.h"

void Student::set_name(const std::string &name_str)
{
    name = name_str;
}

void Student::set_age(int age_val)
{
    age = age_val;
}

void Student::set_pocket_money(double money_val)
{
    pocket_money = money_val;
}
```

This works, but it is not the best long-term structure. `main.cpp` should usually focus on the application logic. The detailed implementation of the `Student` methods should belong in a separate source file.

So the next step is to move these definitions into `student.cpp`.

10. Moving Method Definitions into `student.cpp`

The `student.cpp` file contains the actual implementation of the `Student` methods.

```cpp
#include "student.h"

void Student::set_name(const std::string &name_str)
{
    name = name_str;
}

void Student::set_age(int age_val)
{
    age = age_val;
}

void Student::set_pocket_money(double money_val)
{
    pocket_money = money_val;
}

std::string Student::get_name() const
{
    return name;
}

int Student::get_age() const
{
    return age;
}

double Student::get_pocket_money() const
{
    return pocket_money;
}
```

This creates a clean separation:

`student.h` describes what the `Student` type offers.

`student.cpp` defines how those operations actually work.

`main.cpp` uses the `Student` type to build the application.

This is the same idea used in larger professional C++ projects. The header file gives the interface. The source file contains the implementation.

11. Compiling Multiple `.cpp` Files

When everything was inside one file, we could compile using:

```
g++ main.cpp -o prog
```

But now the program has more than one `.cpp` file.

```
main.cpp
student.cpp
student.h
```

The header file is included by the source files, but it is not normally compiled by itself. The `.cpp` files are compiled and linked together.

The command becomes:

```
g++ main.cpp student.cpp -o prog
```

This tells the compiler:

Compile `main.cpp`.

Compile `student.cpp`.

Link the compiled pieces together.

Produce the final executable called `prog`.

If we forget to include `student.cpp` in the compilation command, the compiler may know that the methods exist because it has seen their declarations in `student.h`, but the linker will not find their actual definitions. This usually results in linker errors.

So, once method definitions are moved into a separate `.cpp` file, that `.cpp` file must also be included in the build command.

12. Giving the Header File to Another Programmer

One major advantage of this structure is that another programmer does not need to see the full implementation in order to use the `Student` type.

They can be given the header file:

```
struct Student
{
  public:
    Student() = default;

    void set_name(const std::string &name_str);
```

```
    void set_age(int age_val);
    void set_pocket_money(double money_val);

    std::string get_name() const;
    int get_age() const;
    double get_pocket_money() const;

  private:
    std::string name{};
    int age{};
    double pocket_money{};
};
```

From this, they know how to create a `Student` object and which methods they are allowed to call.

They do not need to know exactly how `set_age()` is implemented internally. They only need to know that the method exists and what argument it expects.

This is extremely useful in real software development. One team can design and implement the `Student` type, while another team can use it in their application based only on the header file and the compiled implementation.

13. Public Methods as a Design Decision

The public methods are not random. They represent deliberate design decisions.

For example, we may decide to provide:

```
void set_name(const std::string &name_str);
std::string get_name() const;
```

This means outside code is allowed to set and read the name.

But we may decide not to provide a direct setter for some other value. For example, imagine a student has an internal scholarship score. We may allow outside code to read it, but not directly change it.

```
double get_scholarship_score() const;
```

If there is no setter, outside code cannot directly modify that value.

This is a business and design decision. The programmer decides what should be visible, what should be changeable, and what should remain controlled inside the type.

This is not only about syntax. It is about protecting the meaning of the program.

14. Why This Structure Improves the Program

This structure improves the program in several ways.

First, the data is protected from careless direct changes. The member variables are private, so outside code cannot modify them without going through the provided methods.

Second, the code becomes easier to understand. `main.cpp` shows what the program is doing, while `student.cpp` contains the details of how the student operations work.

Third, the project becomes easier to maintain. If the implementation of `set_age()` changes later, the code using the `Student` type may not need to change at all.

Fourth, the header file provides a clean agreement between the type designer and the type user. The user knows what they can call, and the designer remains free to change the internal implementation carefully.

Finally, the program begins to look like professional C++ code. Instead of placing everything inside `main()`, we now have separate responsibilities:

```
student.h     -> declaration of the Student type
student.cpp   -> implementation of Student methods
main.cpp      -> application code using Student
```

15. Complete Flow of the Lesson

The lesson begins with a simple `Student` structure that groups related data.

Then we notice that direct access to member variables can be unsafe because outside code can modify the data freely.

To prevent this, we move the member variables under `private`.

Once the data is private, `main()` can no longer access it directly. So we introduce setter functions to write values and getter functions to read values.

Then we introduce a default constructor so that a newly created student object begins with known values.

Next, we show how modern C++ can use `= default` to ask the compiler to generate the default constructor. For built-in members such as `int` and `double`, we should use in-class initialisers if we want guaranteed initial values.

After that, we separate the declaration of the `Student` type into `student.h`.

Then we first show that method definitions can be written outside the structure using the `Student::` scope resolution syntax.

Finally, we move those method definitions into `student.cpp`, leaving `main.cpp` to use the type cleanly.

The compilation command also changes because the program now has more than one source file:

```
g++ main.cpp student.cpp -o prog
```

This completes the journey from a simple structure to a cleaner, safer, and more professional multi-file C++ program.